**DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE**

CO3- ACTIVITY

# Social_Media_Impact_on_Mental Health

Data Card | Code (14) | Discussion (0) | Suggestions (0)

## 📊 Dataset Overview

| Property | Details |
|---|---|
| Rows | 1,200 Teenagers |
| Columns | 13 Features |
| Missing Values | None |
| File Format | CSV |
| Target Variable | `depression_label` |
| Domain | Mental Health & Social Media Analytics |

# DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE

## 🗂️ Column Descriptions

### Participant Information

| Column | Type | Description |
|---|---|---|
| Teen_ID | Integer | Unique identifier assigned to each participant |
| Age | Integer | Age of the teenager (13–19 years) |
| Gender | Categorical | Gender of the participant (Male/Female) |

### Social Media Usage

| Column | Type | Description |
|---|---|---|
| Social_Media_Hours | Float | Average daily hours spent on social media |
| Preferred_Platform | Categorical | Primary social media platform used (Instagram, TikTok, Both) |

---

### Lifestyle Factors

| Column | Type | Description |
|---|---|---|
| Sleep_Hours | Float | Average daily sleep duration |
| Physical_Activity_Level | Integer | Physical activity frequency or engagement score |

# DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE

### Academic Performance

| Column | Type | Description |
|---|---|---|
| Academic_Performance | Float | Academic achievement or GPA-related performance score |

---

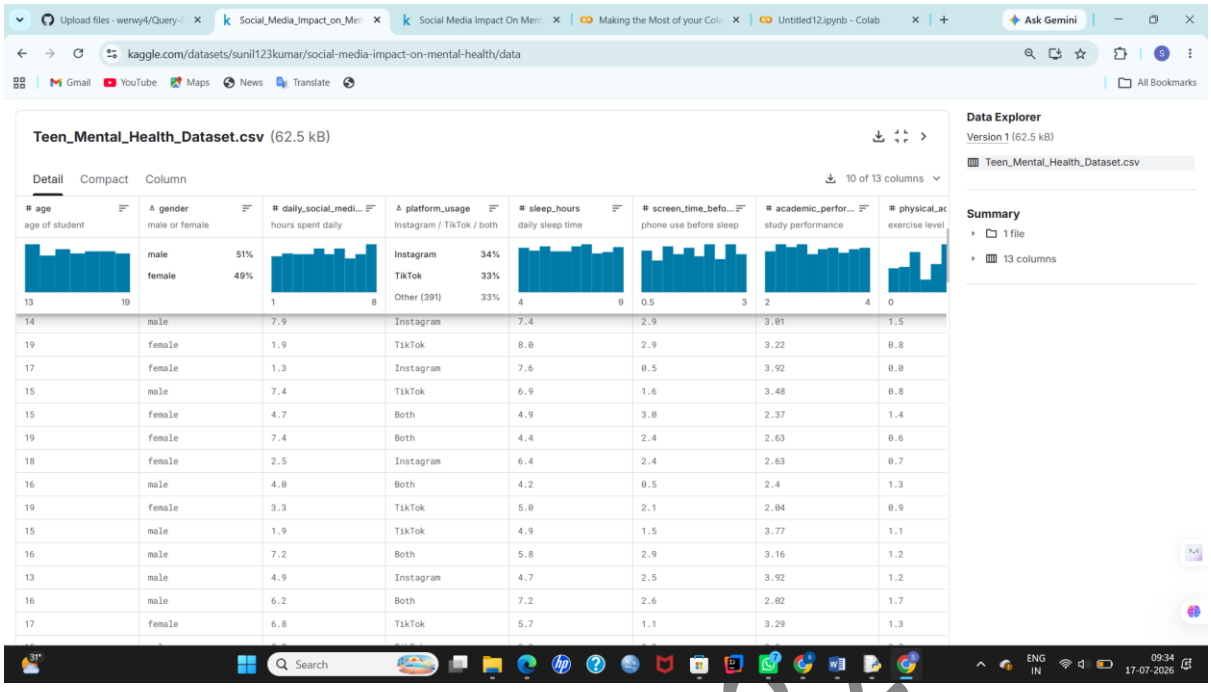
### 🧠 Mental Health Indicators

| Column | Type | Description |
|---|---|---|
| Stress_Level | Integer | Self-reported stress level (1–10 scale) |
| Anxiety_Level | Integer | Self-reported anxiety level (1–10 scale) |
| Depression_Label | Binary | Depression status (0 = Not Depressed, 1 = Depressed) |

### Suggested Target Variables

| Goal | Target Column |
|---|---|
| Classification | Depression_Label |
| Analysis | Stress_Level |
| Analysis | Anxiety_Level |
| Analysis | Social_Media_Hours |
| Research | Sleep_Hours |

# DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE




Code:

1.Handling Missing Values:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Teen_Mental_Health_Dataset.csv")
df.head()
df.tail(5)
df.sample(5)
```

| | age | gender | daily_social_media_hours | platform_usage | sleep_hours | screen_time_before_sleep | academic_performance | physical_activity | social_interaction_level | stress_level | anxiety_level | ad |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 14 | male | 7.9 | Instagram | 7.4 | 2.9 | 3.01 | 1.5 | low | 2 | 2 | |
| 1 | 19 | female | 1.9 | TikTok | 8.0 | 2.9 | 3.22 | 0.8 | high | 8 | 1 | |
| 2 | 17 | female | 1.3 | Instagram | 7.6 | 0.5 | 3.92 | 0.0 | high | 2 | 4 | |
| 3 | 15 | male | 7.4 | TikTok | 6.9 | 1.6 | 3.48 | 0.8 | medium | 1 | 7 | |
| 4 | 15 | female | 4.7 | Both | 4.9 | 3.0 | 2.37 | 1.4 | medium | 3 | 5 | |

| | age | gender | daily_social_media_hours | platform_usage | sleep_hours | screen_time_before_sleep | academic_performance | physical_activity | social_interaction_level | stress_level | anxiety_level |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1195 | 18 | female | 6.8 | Instagram | 6.6 | 2.0 | 2.76 | 1.0 | low | 3 | 4 |
| 1196 | 16 | male | 2.3 | Both | 8.0 | 1.9 | 2.12 | 0.4 | high | 7 | 4 |
| 1197 | 14 | female | 1.7 | Both | 8.7 | 0.7 | 3.98 | 0.8 | high | 1 | 1 |
| 1198 | 15 | male | 3.9 | Both | 8.5 | 2.1 | 3.19 | 0.6 | high | 7 | 9 |
| 1199 | 16 | female | 4.7 | TikTok | 6.5 | 1.0 | 2.91 | 0.9 | medium | 5 | 7 |

```
df.info()
```

```
Data columns (total 13 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   age                        1200 non-null   int64
 1   gender                     1200 non-null   object
 2   daily_social_media_hours   1200 non-null   float64
 3   platform_usage             1200 non-null   object
 4   sleep_hours                1200 non-null   float64
 5   screen_time_before_sleep   1200 non-null   float64
 6   academic_performance       1200 non-null   float64
 7   physical_activity          1200 non-null   float64
 8   social_interaction_level   1200 non-null   object
 9   stress_level               1200 non-null   int64
 10  anxiety_level              1200 non-null   int64
 11  addiction_level            1200 non-null   int64
 12  depression_label           1200 non-null   int64
dtypes: float64(5), int64(5), object(3)
memory usage: 122.0+ KB
```

```
df.duplicated().sum()
df.describe()
```

| | age | daily_social_media_hours | sleep_hours | screen_time_before_sleep | academic_performance | physical_activity | stress_level | anxiety_level | addiction_level | depression_label |
|---|---|---|---|---|---|---|---|---|---|---|
| count | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 | 1200.000000 |
| mean | 15.928333 | 4.536667 | 6.449417 | 1.740333 | 2.990383 | 1.014500 | 5.445833 | 5.636667 | 5.565000 | 0.025833 |
| std | 2.021947 | 2.029599 | 1.442677 | 0.716660 | 0.576758 | 0.582185 | 2.903290 | 2.859453 | 2.830627 | 0.158704 |
| min | 13.000000 | 1.000000 | 4.000000 | 0.500000 | 2.000000 | 0.000000 | 1.000000 | 1.000000 | 1.000000 | 0.000000 |
| 25% | 14.000000 | 2.800000 | 5.200000 | 1.100000 | 2.500000 | 0.500000 | 3.000000 | 3.000000 | 3.000000 | 0.000000 |
| 50% | 16.000000 | 4.500000 | 6.500000 | 1.800000 | 2.990000 | 1.000000 | 5.000000 | 6.000000 | 6.000000 | 0.000000 |
| 75% | 18.000000 | 6.300000 | 7.600000 | 2.400000 | 3.480000 | 1.500000 | 8.000000 | 8.000000 | 8.000000 | 0.000000 |
| max | 19.000000 | 8.000000 | 9.000000 | 3.000000 | 4.000000 | 2.000000 | 10.000000 | 10.000000 | 10.000000 | 1.000000 |

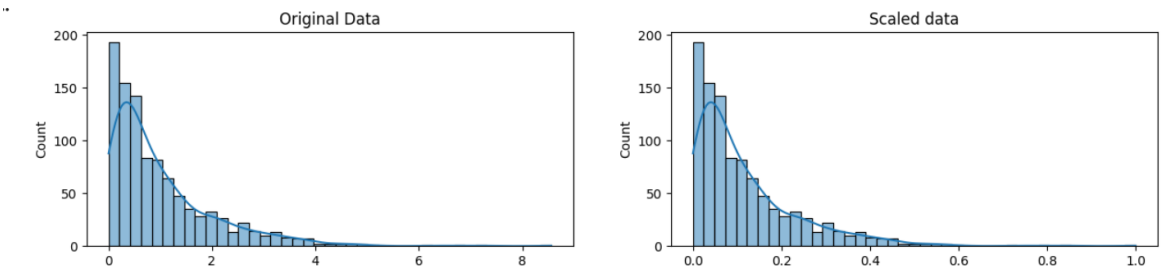
```
df.corr(numeric_only = True)
missing_values_count = df.isnull().sum()
missing_values_count[0:10]
total_cells = np.prod(df.shape)
total_missing = missing_values_count.sum()
```

| | 0 |
|---|---|
| age | 0 |
| gender | 0 |
| daily_social_media_hours | 0 |
| platform_usage | 0 |
| sleep_hours | 0 |
| screen_time_before_sleep | 0 |
| academic_performance | 0 |
| physical_activity | 0 |
| social_interaction_level | 0 |
| stress_level | 0 |
| anxiety_level | 0 |
| addiction_level | 0 |
| depression_label | 0 |

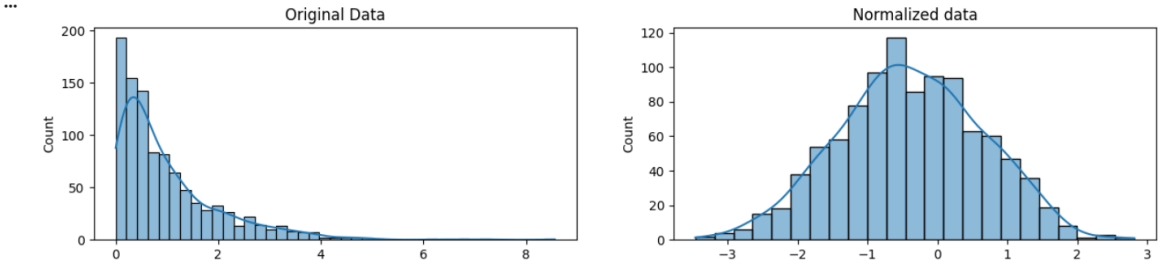
```
# percent of data that is missing
percent_missing = (total_missing/total_cells) * 100
print(percent_missing)
```

**2.Scaling and Normalisation**

```
import pandas as pd
import numpy as np
from scipy import stats
from mlxtend.preprocessing import minmax_scaling
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(0)
original_data = np.random.exponential(size=1000)

# mix-max scale the data between 0 and 1
scaled_data = minmax_scaling(original_data, columns=[0])

# plot both together to compare
fig, ax = plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(scaled_data, ax=ax[1], kde=True, legend=False)
ax[1].set_title("Scaled data")
plt.show()
normalized_data = stats.boxcox(original_data)
```




```
# plot both together to compare
fig, ax=plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(normalized_data[0], ax=ax[1], kde=True, legend=False)
ax[1].set_title("Normalized data")
plt.show()
```



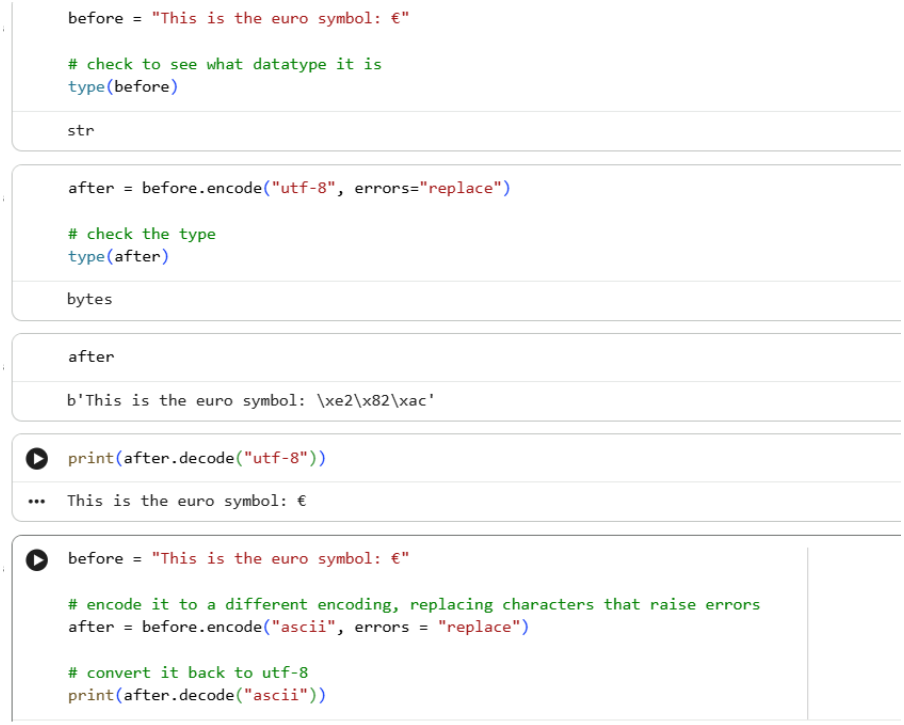

**3.Parsing Dates**

```
#parsing data
import pandas as pd
import numpy as np
import seaborn as sns
import datetime
# set seed for reproducibility
np.random.seed(0)

sns.countplot(
    data = df,
    x = "depression_label",

)
num_cols = df.select_dtypes(include = ["int64",'float']).columns
plt.figure(figsize = (15,10))

for i,col in enumerate(num_cols,1):
    plt.subplot(4,3,i)
    sns.histplot(df[col],kde = True)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm",
fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
cat_cols = df.select_dtypes(include = ["object","category"]).columns
for col in cat_cols:
    display(
        df[col].value_counts().rename_axis(col).reset_index(name =
"Count")
    )
```

**4.Character encodings**

```
import charset_normalizer
before = "This is the euro symbol: €"
type(before)
after = before.encode("utf-8", errors="replace")
type(after)
after
print(after.decode("utf-8"))
before = "This is the euro symbol: €"
errors
after = before.encode("ascii", errors = "replace")
print(after.decode("ascii"))
```




**5.Inconsistent data entry**

```
import pandas as pd
import numpy as np
```

# DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE

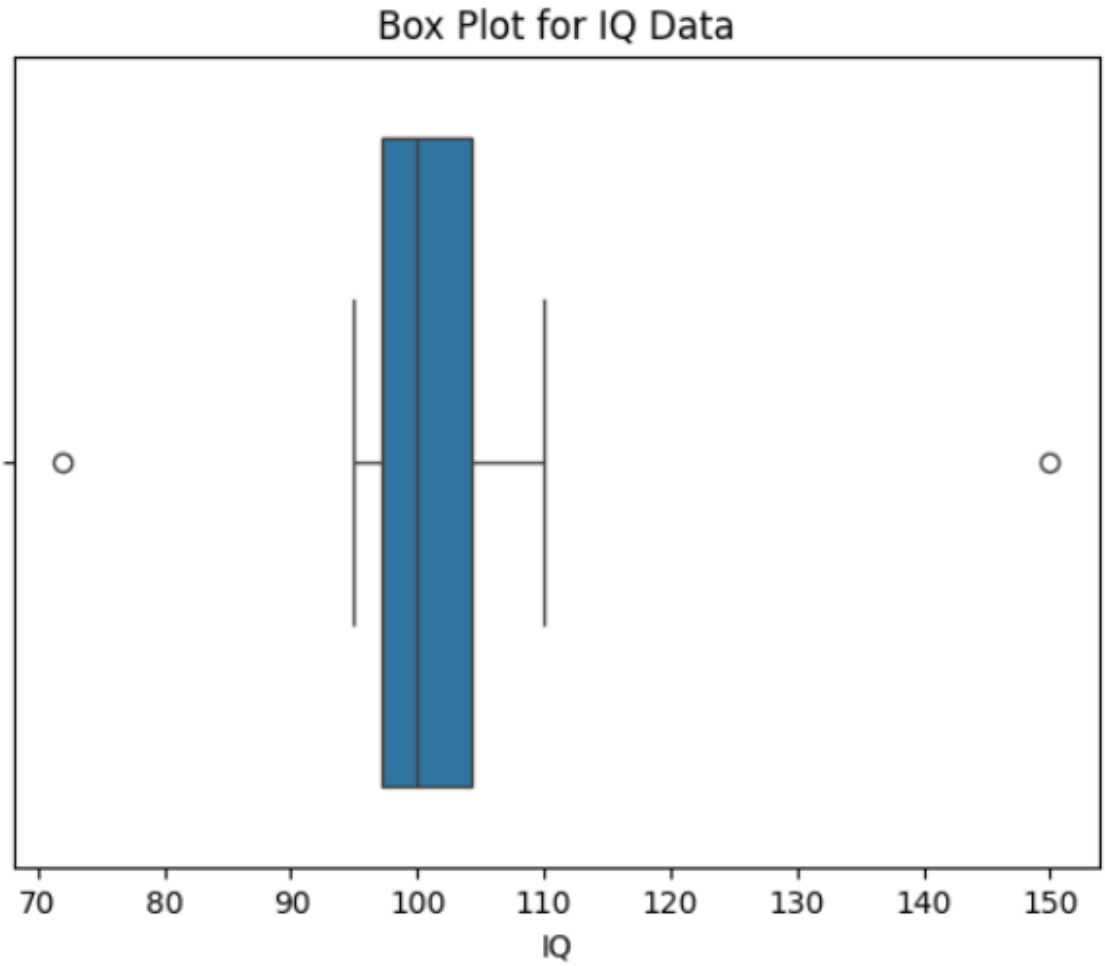
```
!pip install fuzzywuzzy
import fuzzywuzzy
from fuzzywuzzy import process
import charset_normalizer
professors = pd.read_csv("/content/pakistan_intellectual_capital-
selected-columns.csv")
np.random.seed(0)
countries = professors['Country'].unique()
countries.sort()
countries
professors['Country'] = professors['Country'].str.lower()
professors['Country'] = professors['Country'].str.strip()
countries = professors['Country'].unique()
countries.sort()
countries
matches = fuzzywuzzy.process.extract("south korea", countries,
limit=10, scorer=fuzzywuzzy.fuzz.token_sort_ratio)
matches
def replace_matches_in_column(df, column, string_to_match, min_ratio =
47)
    strings = df[column].unique()
    matches = fuzzywuzzy.process.extract(string_to_match, strings,
                                         limit=10,
scorer=fuzzywuzzy.fuzz.token_sort_ratio)
    close_matches = [matches[0] for matches in matches if matches[1] >=
min_ratio]
    rows_with_matches = df[column].isin(close_matches)
matches
    df.loc[rows_with_matches, column] = string_to_match
    print("All done!")
replace_matches_in_column(df=professors, column='Country',
string_to_match="south korea")
countries = professors['Country'].unique()
countries.sort()
countries
```

| | Unnamed: 0 | S# | Teacher Name | University Currently Teaching | Department | Province University Located | Designation | Terminal Degree | Graduated from | Country |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 2 | 3 | Dr. Abdul Basit | University of Balochistan | Computer Science & IT | Balochistan | Assistant Professor | PhD | Asian Institute of Technology | Thailand |
| 1 | 4 | 5 | Dr. Waheed Noor | University of Balochistan | Computer Science & IT | Balochistan | Assistant Professor | PhD | Asian Institute of Technology | Thailand |
| 2 | 5 | 6 | Dr. Junaid Baber | University of Balochistan | Computer Science & IT | Balochistan | Assistant Professor | PhD | Asian Institute of Technology | Thailand |
| 3 | 6 | 7 | Dr. Maheen Bakhtyar | University of Balochistan | Computer Science & IT | Balochistan | Assistant Professor | PhD | Asian Institute of Technology | Thailand |
| 4 | 24 | 25 | Samina Azim | Sardar Bahadur Khan Women's University | Computer Science | Balochistan | Lecturer | BS | Balochistan University of Information Technolo... | Pakistan |

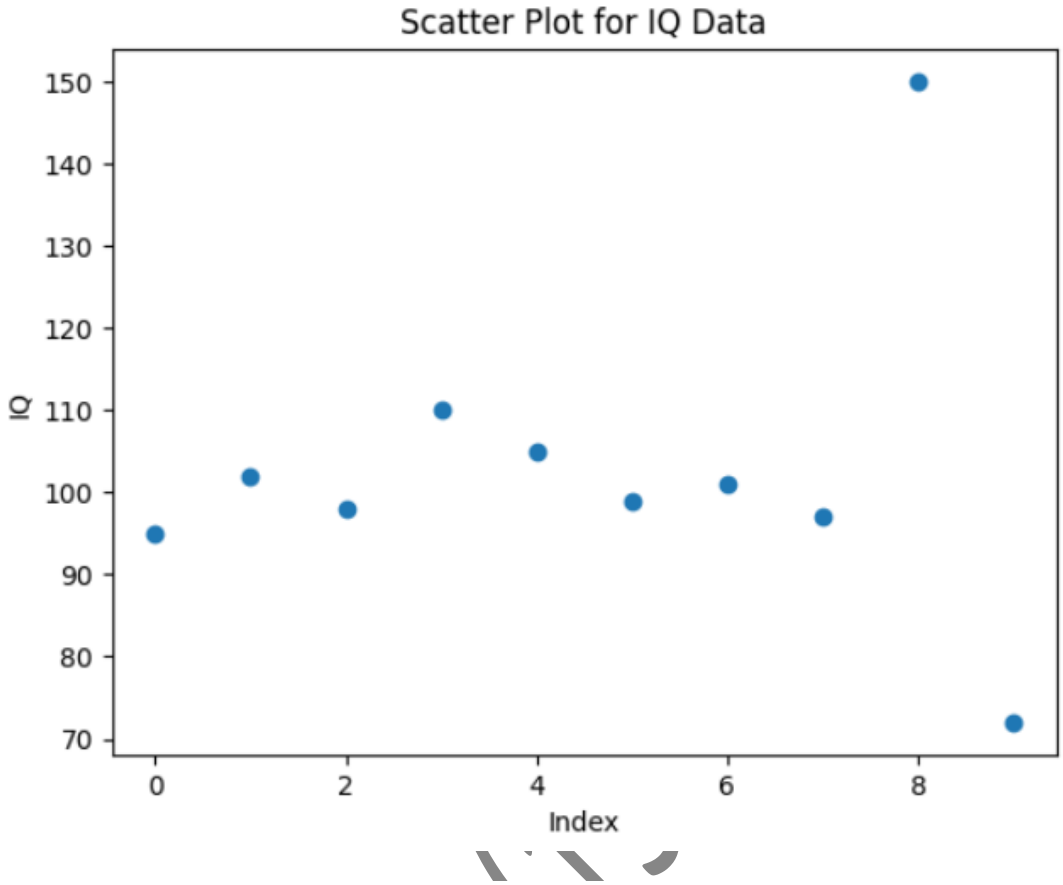
6. **outliers**

```
import pandas as pd
```

```
import seaborn as sns
import matplotlib.pyplot as plt

data = {"IQ": [95, 102, 98, 110, 105, 99, 101, 97, 150, 72]}
df = pd.DataFrame(data)

sns.boxplot(x=df["IQ"])
plt.title("Box Plot for IQ Data")
plt.show()
```




```
plt.scatter(df.index, df["IQ"])
plt.xlabel("Index")
plt.ylabel("IQ")
plt.title("Scatter Plot for IQ Data")
plt.show()
```




```
import pandas as pd
import numpy as np
from scipy.stats import zscore

np.random.seed(0)
normal_iq = np.random.normal(100, 5, 100)
outliers = [30, 250]

iq_data = np.concatenate([normal_iq, outliers])

df = pd.DataFrame({"IQ": iq_data})
df["Z_Score"] = zscore(df["IQ"])

outliers_z = df[np.abs(df["Z_Score"]) > 3]
print(outliers_z)
```

output:

```
        IQ   Z_Score
100   30.0 -4.156281
101  250.0  8.708296
```

```
Q1 = df["IQ"].quantile(0.25)
Q3 = df["IQ"].quantile(0.75)
```

```
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers_iqr = df.query("IQ < @lower_bound or IQ > @upper_bound")
print(outliers_iqr)


from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=5, min_samples=2)
df["Cluster"] = dbscan.fit_predict(df[["IQ"]])

outliers_dbscan = df[df["Cluster"] == -1]
print(outliers_dbscan)
```

output:

```
IQ   Z_Score  Cluster
100   30.0 -4.156281       -1
101  250.0  8.708296       -1
```

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.2, random_state=42)
df["Outlier"] = iso.fit_predict(df[["IQ"]])

outliers_iso = df[df["Outlier"] == -1]
print(outliers_iso)
```

```
...           IQ   Z_Score  Cluster  Outlier
0     108.820262  0.452761        0       -1
3     111.204466  0.592178        0       -1
4     109.337790  0.483024        0       -1
11    107.271368  0.362189        0       -1
20     87.235051 -0.809441        0       -1
24    111.348773  0.600617        0       -1
25     92.728172 -0.488229        0       -1
28    107.663896  0.385142        0       -1
33     90.096018 -0.642145        0       -1
41     92.899910 -0.478187        0       -1
42     91.468649 -0.561880        0       -1
43    109.753877  0.507355        0       -1
46     93.736023 -0.429295        0       -1
48     91.930511 -0.534873        0       -1
63     91.368587 -0.567731        0       -1
66     91.849008 -0.539639        0       -1
83     92.318782 -0.512169        0       -1
85    109.479446  0.491307        0       -1
97    108.929352  0.459140        0       -1
100    30.000000 -4.156281       -1       -1
101   250.000000  8.708296       -1       -1
```

**7.Duplicates removal**

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'David'],
    'Age': [25, 30, 25, 35, 30, 40],
    'City': ['New York', 'Los Angeles', 'New York', 'Chicago', 'Los
Angeles', 'San Francisco']
}

df = pd.DataFrame(data)
df
```

| | Name | Age | City |
|---|---|---|---|
| 0 | Alice | 25 | New York |
| 1 | Bob | 30 | Los Angeles |
| 2 | Alice | 25 | New York |
| 3 | Charlie | 35 | Chicago |
| 4 | Bob | 30 | Los Angeles |
| 5 | David | 40 | San Francisco |

```
duplicates = df.duplicated()

duplicates
```

| | |
|---|---|
| 0 | False |
| 1 | False |
| 2 | True |
| 3 | False |
| 4 | True |
| 5 | False |

dtype: bool

```
df_no_duplicates_columns = df.drop_duplicates(subset=['Name', 'City'])
(df_no_duplicates_columns)
```

| | Name | Age | City |
|---|---|---|---|
| 0 | Alice | 25 | New York |
| 1 | Bob | 30 | Los Angeles |
| 3 | Charlie | 35 | Chicago |
| 5 | David | 40 | San Francisco |

```
df_keep_last = df.drop_duplicates(keep='last')
(df_keep_last)
```

| | Name | Age | City |
|---|---|---|---|
| 2 | Alice | 25 | New York |
| 3 | Charlie | 35 | Chicago |
| 4 | Bob | 30 | Los Angeles |
| 5 | David | 40 | San Francisco |



**8.Fuzzy matching**

```
# Install openpyxl (if needed)
!pip install openpyxl

import pandas as pd
import sqlite3

# Upload the Excel file
from google.colab import files
uploaded = files.upload()

# Read the Excel file
df = pd.read_excel("FirstName_LastName_Data.xlsx")

# Rename columns to match your SQL query
df.columns = ['firstName', 'lastName']

# Create SQLite connection
conn = sqlite3.connect(':memory:')

# Register SOUNDEX function (SQLite already supports it)
df.to_sql('load', conn, index=False, if_exists='replace')

# Execute SQL query
```

# DSAO5O4-QUERY PROCESSING FOR DATA SCIENCE

```
query = """
SELECT DISTINCT
    ss.firstName,
    ss.lastName,
    sd.firstName,
    sd.lastName
FROM load AS ss, load AS sd
WHERE ss.firstName = sd.firstName
  AND SOUNDEX(ss.lastName) = SOUNDEX(sd.lastName)
  AND substr(ss.lastName,1,2) = substr(sd.lastName,1,2)
  AND ss.lastName <> sd.lastName;
"""

result = pd.read_sql_query(query, conn)

# Display result
print(result)
```

Choose Files FirstName_...e_Data.xlsx

**FirstName_LastName_Data.xlsx**(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 5106 bytes, last modified: 7/24/2026 - 100% d

```
Saving FirstName_LastName_Data.xlsx to FirstName_LastName_Data (1).xlsx
  firstName           lastName firstName           lastName
0  Benjamin            Sheriff  Benjamin            Sherrif
1  Benjamin            Sherrif  Benjamin            Sheriff
2  Virginia      Pulley Alberts  Virginia      Pulley-Alberts
3  Virginia      Pulley-Alberts  Virginia      Pulley Alberts
4   Rodrigo             Delgado   Rodrigo          Delgado Jr
5   Rodrigo          Delgado Jr   Rodrigo             Delgado
6  Leonardo            Madrigal  Leonardo  Madrigal Del Valle
7  Leonardo  Madrigal Del Valle  Leonardo            Madrigal
```